AIntegrity v4.0: Technical Specification for a Multimodal, Verifiable AI Governance Architecture

Introduction: The Imperative for Verifiable AI Integrity

The AIntegrity framework is a neuro-symbolic architecture conceived as a computational engine for critical thought, designed to automate the rigorous process of rational validation for outputs generated by Artificial Intelligence (AI) systems. Its philosophical underpinnings are rooted in a synthesis of two of the 20th century's most profound scientific methodologies: the empirical skepticism of Richard Feynman and the axiomatic rationalism of Albert Einstein. This combination creates a powerful dialectic between inductive pattern recognition and deductive logical verification, which is mirrored in the system's architecture. The Large Language Model (LLM) components act as the inductive "Guesser" (Feynman), trained on vast corpora to recognize patterns and form hypotheses about the logical structure of an input. Complementing this is the symbolic core—formal verification tools like Satisfiability Modulo Theories (SMT) solvers and Zero-Knowledge Proofs (ZKPs)—which function as the deductive "Theorist" (Einstein), rigorously testing these hypotheses against the unyielding axioms of mathematical logic.

The urgent need for the AIntegrity v4.0 architecture is driven by three fundamental shifts in the AI landscape. First, the expansion of AI beyond text into **multimodal domains** means that systems now interpret and generate images, audio, and video, creating a vastly larger surface for potential failures and adversarial attacks. Second, the increasing **autonomy and agency** of AI systems, which can now execute tasks and make decisions with real-world consequences, transforms unverifiable outputs from a data quality issue into a direct operational and safety risk. Finally, the **codification of AI accountability** through landmark regulatory frameworks has moved AI ethics from a philosophical debate to a legal and operational mandate. The full implementation of regulations like the European Union's AI Act, particularly its stringent obligations for General-Purpose AI (GPAI) models effective August 2025, and the widespread adoption of the NIST AI Risk Management Framework (RMF), demand a new class of tools capable of providing continuous, verifiable assurance.

This technical specification details a paradigm shift from AIntegrity v2.1 to v4.0. The previous version focused on *post-hoc forensic integrity*—proving cryptographically that a log of an AI interaction was not tampered with after the fact. AIntegrity v4.0 elevates this concept to *proactive, real-time, and privacy-preserving computational integrity*. The goal is no longer just to secure the record, but to securely verify the computation itself as it happens, proving that an AI's decision was made correctly, by an authorized model, and in compliance with established policies, all without compromising sensitive user data or the model's intellectual property.

Part I: AIntegrity v4.0 - A Unified Architecture for Multimodal, Verifiable AI Governance

The AIntegrity v4.0 architecture is a comprehensive, multi-stage pipeline designed to systematically ingest, deconstruct, verify, and monitor AI interactions across multiple modalities. It operates as a Directed Acyclic Graph (DAG), ensuring a logical and traceable flow of analysis where the output of one module serves as the input for subsequent stages. The high-level architecture integrates the established neuro-symbolic core of previous versions with new components for multimodal analysis, dynamic trust scoring, real-time adversarial monitoring, and a decentralized, privacy-preserving ledger that serves as the system's immutable root of trust.

The core principles guiding the v4.0 design are:

- **Verifiability by Design**: Every component is architected not merely to produce an output, but to generate a verifiable *proof* of its output's integrity and correctness. This principle moves beyond simple logging to active, computational verification.
- **Privacy by Default**: The system leverages advanced cryptographic techniques such as Zero-Knowledge Proofs (ZKPs) and Trusted Execution Environments (TEEs) to ensure that verification processes do not require exposing sensitive user data, proprietary model information, or confidential business logic. This aligns directly with the "Data Protection by Design and by Default" principle mandated by Article 25 of the GDPR.
- **Decentralized Trust**: The architecture's foundation shifts from a centralized, single-authority log to a distributed, cryptographically secured ledger. This mitigates single points of failure and control, creating a more resilient and trustworthy audit trail that is not reliant on the integrity of a single entity.
- **Continuous Assurance**: The framework moves away from periodic or post-hoc audits towards a model of real-time, continuous monitoring and risk management. This allows for the immediate detection of and response to adversarial attacks, performance degradation, and compliance violations.

The following table provides a comprehensive overview of all modules within the AIntegrity v4.0 framework, categorized by function and highlighting components that are new or have been significantly updated from version 2.1.

Module Name	Version	Primary Function	Key Dependencies	Status
Core Ingestion & Logging				
TranscriptProcessor	2.0	Ingests and structures raw multimodal conversation transcripts for analysis.	spaCy, OpenCV	Updated
VerifiableInteractionLedger	4.0	Provides a decentralized, tamper-proof, and privacy-preserving audit trail using blockchain, ZKPs, and TEEs.	Hyperledger Fabric, Circom, Intel SGX SDK	**New**
Core Analysis Modules				
PLIEngine	4.0	Performs	TranscriptProcess	Updated

Module Name	Version	Primary Function	Key Dependencies	Status
		neuro-symbolic logical analysis, fallacy detection, and formal verification on textual claims.	or, z3-solver, LogicLLaMA	
VisualConsistency Verifier	1.0	Assesses semantic consistency between images and associated text using CLIP-based models.	TranscriptProcess or, CLIP	**New**
MediaIntegrityAss essor	1.0	Verifies the integrity of image, video, and audio inputs using perceptual hashing.	TranscriptProcess or, pHash	**New**
ComplianceScanM odule	2.0	Scans content for legal, regulatory, and policy violations (e.g., GDPR, PII, hate speech).	TranscriptProcess or, Presidio	Updated
CitationVerifier	4.0	Verifies the validity, format, and factual support of citations in AI output, including DOI and URL resolution.	TranscriptProcess or, NLI models	Updated
Behavioral & Trust Modules				
SessionDriftDetect or	4.0	Monitors for semantic and factual contradictions across an entire conversation session.	TranscriptProcess or, sentence-transfor mers	Updated
AdversarialThreat Monitor	1.0	Provides real-time detection of adversarial attacks, prompt	All inputs/outputs, SHAP	**New**

Module Name	Version	Primary Function	Key Dependencies	Status
		injections, and anomalous behavior using drift metrics and XAI.		
TrustGradingEngine	4.0	Calculates a dynamic, multi-dimensional trust score with a temporal decay model.	All analysis modules	Updated
Remediation & Enforcement				
ReconstructionAdvisor	2.0	Generates actionable suggestions for resolving detected inconsistencies or improving AI outputs.	PLIEngine, SessionDriftDetector	Updated
SentinelEnforcementCore	2.0	Consolidates all findings to enforce final safety and compliance guardrails (e.g., halt, flag, alert).	All analysis modules	Updated
Governance & Reporting				
RegulatoryComplianceMapper	1.0	Maps audit findings and system features to specific requirements of the EU AI Act and NIST AI RMF.	All analysis modules, VerifiableInteractionLedger	**New**
AuditTraceVisualizer	2.0	Produces visual representations (graphs, timelines) of the verifiable audit trail.	VerifiableInteractionLedger	Updated

Part II: The Cryptographic Bedrock - Verifiable Interaction Ledger (VIL) v4.0

The foundation of AIntegrity v4.0 is the Verifiable Interaction Ledger (VIL), a significant architectural evolution from the Verifiable Interaction Logging (VIL) Engine of previous versions. Where the VIL Engine focused on creating a tamper-evident, centralized log file , the VIL v4.0 is

a decentralized, append-only, and privacy-preserving ledger designed to serve as an unimpeachable root of trust for all AI interactions and system operations. Its design is predicated on the principles of durable, append-only logs, which ensure that data, once written, cannot be altered, providing a permanent and verifiable history of events.

Blockchain-Based Trust Anchors

To achieve decentralized and immutable storage, the VIL is built upon a permissioned blockchain network. This choice is critical for enterprise applications, where control over network participation and data confidentiality are paramount.

Protocol Selection: Hyperledger Fabric

A permissioned blockchain is superior to a public, permissionless one (like the main Ethereum network) for enterprise audit trails. Public blockchains are, by design, transparent to all participants, which directly conflicts with the need to protect proprietary business data, sensitive user information under GDPR, and the AI model's intellectual property. Furthermore, public chains often suffer from lower transaction throughput and unpredictable costs (gas fees), making them unsuitable for the high-volume, real-time logging required by AIntegrity. Hyperledger Fabric, a project hosted by the Linux Foundation, is selected as the underlying framework for the VIL. It provides a modular architecture that excels in the areas critical for this use case: privacy through "channels" that allow for confidential transactions between specific participants, higher performance due to its pluggable consensus mechanisms, and a permissioned governance model that aligns with enterprise compliance and control requirements. The following table justifies this selection.

Feature	Hyperledger Fabric	Ethereum (Public L1/L2)	Certificate Transparency	Rationale for AIntegrity v4.0
Permissioning	Permissioned (Known Identities)	Permissionless (Anonymous)	Public (Known CAs)	Essential for enterprise accountability. Only authorized nodes (e.g., internal audit, regulators) can join the network.
Privacy	High (Private Channels)	Low (Public Ledger)	Public by Design	Critical for protecting sensitive user data (GDPR) and proprietary model information. Channels can isolate logs for different clients or use cases.
Throughput	High (e.g., >2000 TPS)	Low to Medium (Varies by L2)	N/A (Not a transaction	Necessary for real-time logging

Feature	Hyperledger Fabric	Ethereum (Public L1/L2)	Certificate Transparency	Rationale for AIntegrity v4.0
			system)	of high-frequency AI interactions without becoming a bottleneck.
Transaction Costs	Low / Predictable	High / Volatile (Gas Fees)	N/A	Enterprise systems require predictable operational costs, which volatile gas fees on public networks preclude.
Governance	Consortium-based	Decentralized Community	Browser/CA Forum	A defined governance model allows enterprises to set and enforce policies for the audit trail itself.
Suitability	**Excellent**	Poor	Unsuitable	Fabric provides the optimal balance of immutability, privacy, performance, and control for enterprise-grade, verifiable AI audit trails.

On-Chain Data Structure and Smart Contracts

Every auditable action within the AIntegrity system is recorded as a transaction on the Hyperledger Fabric ledger. The data payload for each transaction is a JSON object representing an AuditEvent.
The AuditEvent structure is defined as follows:

```
{
  "eventId": "string (UUID)",
  "sessionId": "string (UUID)",
  "timestampUTC": "string (ISO 8601)",
  "eventType": "string (e.g., INPUT, ANALYSIS, TRUST_GRADING)",
  "actorId": "string (Decentralized Identifier for user/agent)",
  "contentHash": "string (SHA-256 hash of the event content)",
  "prevEventHash": "string (SHA-256 hash of the previous event's
on-chain record)",
  "proofPayload": {
    "proofType": "string (e.g., 'ZKP_SNARK', 'TEE_ATTESTATION')",
    "proofData": "object (The ZKP or TEE attestation data)"
```

```
    }
}
```

A **smart contract** (or "chaincode" in Fabric terminology) governs all interactions with the ledger. This chaincode, written in a general-purpose language like Go or Node.js, enforces the integrity rules of the audit trail. Its key functions include:

- logEvent(eventDetails): This function accepts a new AuditEvent. It verifies that the prevEventHash matches the hash of the last recorded event in that session, enforcing a strict, unbreakable chronological chain. It also validates the structure of the proofPayload before committing the new event to the ledger.
- getEvent(eventId): Retrieves a specific event record from the ledger.
- verifyEventProof(eventId): This function takes an eventId and triggers an on-chain or off-chain verification process for the associated proofPayload. For a ZKP, it would run the verifier logic against the proof data. For a TEE attestation, it would check the signature against the hardware vendor's public keys.

Privacy-Preserving Verifiable Computation

A key innovation in v4.0 is the ability to not just record that an analysis was performed, but to prove that it was performed *correctly* and *privately*. This is achieved by combining ZKPs and TEEs, which are complementary technologies addressing different aspects of the verifiability challenge. ZKPs are ideal for proving the correctness of auditable, rule-based logic without revealing the inputs, while TEEs are suited for proving the integrity of the execution environment for proprietary, "black-box" AI models.

Zero-Knowledge Proofs (ZKPs) for Verifiable Logic

For deterministic modules where the logic must be auditable, such as the ComplianceScanModule, zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) are employed. This allows AIntegrity to prove, for example, that a block of text was scanned for PII and that all detected PII was redacted, without revealing the PII itself to the blockchain or an auditor.
The implementation follows the **Circom and snarkjs** workflow :

1. **Circuit Design (Circom)**: The compliance rule is encoded as an arithmetic circuit. For a PII redaction check, the circuit would take a public input (the hash of the original text) and a private input, or "witness" (the original text and the list of detected PII entities). The circuit's logic would verify that hashing the redacted text produces a different, expected hash, proving the redaction occurred.
2. **Witness Generation**: The ComplianceScanModule executes its logic (e.g., using the Presidio library) and uses the results to generate the witness for the circuit.
3. **Proof Generation (snarkjs)**: The system uses the generated witness and a proving key to create a compact zk-SNARK proof. This proof is a small data object that mathematically attests to the correct execution of the circuit.
4. **On-Chain Verification**: The generated proof is placed in the proofPayload of the AuditEvent and submitted to the VIL. The verifyEventProof function in the smart contract can then run the corresponding zk-SNARK verifier algorithm against the proof, confirming its validity in a decentralized and tamper-proof manner.

Trusted Execution Environments (TEEs) for Confidential Model Execution

For complex, proprietary AI models (e.g., a custom fraud detection model or a fine-tuned LLM in the PLIEngine), creating a ZKP circuit is computationally infeasible. Here, the goal is to protect the model's intellectual property while proving that the specific, authorized version of the model was executed on the input data without tampering. This is achieved using **Trusted Execution Environments (TEEs)**, such as **Intel Software Guard Extensions (SGX)**.
The workflow is as follows:

1. **Enclave Execution**: The proprietary AI model is loaded and executed inside an SGX enclave, a hardware-isolated memory region protected from the host operating system and any other processes.
2. **Remote Attestation**: The enclave generates a cryptographic attestation. This is a data structure signed by the CPU's unique private key, which contains a hash of the code and data loaded into the enclave.
3. **Attestation Verification**: An external party (or the VIL smart contract) can verify this attestation using Intel's public keys. A successful verification proves that a specific, known model binary (identified by its hash) was executed on a genuine Intel SGX-enabled CPU.
4. **Ledger Record**: The generated attestation is placed in the proofPayload of the AuditEvent and logged on the VIL, creating an immutable record that links a specific model execution to a specific input and output.

By combining these two technologies, AIntegrity v4.0 provides a multi-layered verification system that can offer strong, privacy-preserving integrity guarantees across a wide range of AI components, from simple, auditable rules to complex, proprietary models.

Part III: Advanced Analysis and Scoring Modules

AIntegrity v4.0 introduces a suite of new and updated analysis modules to handle the complexities of multimodal inputs, dynamic trust assessment, and real-time adversarial threats. These modules provide the core analytical capabilities that feed into the Verifiable Interaction Ledger.

Multimodal Verification Engine

The architecture expands beyond text-only analysis to address the growing prevalence of multimodal AI systems.

VisualConsistencyVerifier

This module is designed to assess the semantic consistency between images and their associated textual descriptions, a critical task for detecting a model's compositional understanding failures. It is built using a pre-trained **Contrastive Language-Image Pre-training (CLIP)** model. The core mechanism involves:

1. **Embedding Generation**: The module uses a CLIP model (e.g., ViT-L/14) to generate separate vector embeddings for the input image and the input text.
2. **Similarity Scoring**: It then calculates the cosine similarity between the image embedding and the text embedding. A high similarity score indicates strong alignment, while a low

score suggests a potential contradiction, hallucination, or irrelevant pairing.

3. **Addressing Compositional Weakness**: Standard CLIP models are known to exhibit "bag-of-words" behavior, where they might correctly identify objects and attributes but fail to bind them correctly (e.g., confusing "a red square and a blue circle" with "a blue square and a red circle"). To mitigate this, the VisualConsistencyVerifier incorporates advanced techniques like **Linear Attribute Binding CLIP (LABCLIP)**, which applies a learned linear transformation to the text embeddings before the similarity comparison to improve attribute-object binding accuracy. This allows for more nuanced verification of compositional claims.

MediaIntegrityAssessor

This module is responsible for verifying the integrity of media files (images, videos, audio) against manipulation and forgery, such as deepfakes. Unlike cryptographic hashes that change with any single-bit modification, this module uses **perceptual hashing**. A perceptual hash algorithm generates a "fingerprint" of a media file based on its actual content, making it robust to benign alterations like compression, resizing, or format changes, while still being sensitive to significant content modifications.
The process involves:

1. **Feature Extraction**: For images and video frames, the algorithm extracts key visual features. For audio, it first converts the audio signal into a **spectrogram**—a visual representation of the spectrum of frequencies—and then extracts features from this image.
2. **Hash Generation**: These features are then converted into a compact hash value (e.g., using an open-source library like pHash).
3. **Integrity Check**: The generated hash can be compared against a database of known media hashes to detect duplicates or near-duplicates. For an audit trail, the perceptual hash of an input media file is logged on the VIL. If the same content appears later, its hash can be compared to the ledger to verify its origin and detect potential tampering over time.

Dynamic Trust Scoring Engine

AIntegrity v4.0 moves beyond the static, per-session trust scores of previous versions to a dynamic model that reflects the temporal nature of trust. Trust is not a permanent state; it can decay over time in the absence of reinforcing evidence.

The TrustDecayModel

The TrustGradingEngineV4 calculates a multi-dimensional overall_score based on inputs from all analysis modules. However, this score is now subject to a **temporal decay function**. This models the principle that confidence in an AI agent's reliability should decrease if it has not been successfully evaluated recently.
The decay is modeled using a **Logistic function**, which provides a smooth, S-shaped curve representing a gradual loss of trust that levels off at a baseline.
The mathematical formulation is: $T(t) = T_{min} + \frac{T_{max} - T_{min}}{1 + e^{k(t - t_{mid})}}$
Where:

- $T(t)$ is the trust score at time t.

- T_{max} is the maximum trust score, typically set to the overall_score of the last successful interaction.
- T_{min} is the asymptotic minimum trust score, representing a baseline level of distrust or neutrality (e.g., 0.2).
- k is the decay rate constant, which determines how quickly trust erodes.
- t is the time elapsed since the last interaction.
- t_{mid} is the midpoint of the decay, representing the time at which the trust score is decaying most rapidly.

This model is event-driven:

- **Positive Events**: A new interaction that results in a high overall_score resets the clock (t=0) and sets a new T_{max}.
- **Negative Events**: A detected failure (e.g., a contradiction or compliance violation) can result in an immediate penalty to the current trust score and may also increase the decay rate k, reflecting that trust is harder to maintain after a breach.

Real-Time Adversarial Monitoring & Anomaly Detection

To provide proactive defense, the AdversarialThreatMonitor module continuously analyzes the stream of inputs and outputs of the AI system in real-time.

Drift Detection

The module monitors for two primary types of drift, which can indicate either a change in the operating environment or a subtle adversarial attack :

1. **Data Drift (Covariate Shift)**: This occurs when the statistical properties of the input data change. For example, if a customer service bot trained on formal language starts receiving queries in informal slang.
2. **Concept Drift**: This occurs when the relationship between inputs and correct outputs changes. For example, a financial advice bot's recommendations may become invalid after a sudden market shift.

The monitor uses the **Population Stability Index (PSI)** to quantify this drift. It compares the distribution of features in the current data stream against a baseline (e.g., the training data or a stable production period). A PSI value above a predefined threshold (e.g., **PSI > 0.2**) signifies a significant distribution shift, triggering an alert for model retraining or investigation.

XAI-Based Anomaly Detection

For detecting novel or sophisticated adversarial attacks that may not cause significant statistical drift, the monitor employs an eXplainable AI (XAI) based approach. This technique moves beyond simply looking at the input data to analyzing the AI's *internal reasoning* for its response. The process is as follows:

1. **Baseline Feature Importance**: During a baseline period, the system uses a technique like **SHAP (SHapley Additive exPlanations)** to calculate the feature importance values for a representative set of legitimate prompts. This creates a profile of which parts of an input "normally" influence the model's output.
2. **Real-Time Monitoring**: For each incoming prompt, the monitor calculates its SHAP values in real-time.
3. **Anomaly Detection**: It then compares the SHAP value distribution of the new prompt to

the established baseline. An input that is syntactically benign but triggers a highly anomalous internal response (i.e., the model focuses on unusual parts of the prompt) is flagged as a potential adversarial attack, such as an indirect prompt injection. This allows the system to detect attacks based on their effect on the model's reasoning process, even if the attack vector has never been seen before.

Part IV: Regulatory and Governance Alignment

AIntegrity v4.0 is architected to serve as a foundational technology for organizations seeking to comply with the increasingly complex global landscape of AI regulations. It provides the technical mechanisms to meet the stringent requirements of the EU AI Act and to operationalize the principles of the NIST AI Risk Management Framework.

Conformance with the EU AI Act

The platform directly addresses the obligations for providers of General-Purpose AI (GPAI) models, which came into effect in August 2025.

- **Technical Documentation and Record-Keeping (Article 53)**: The EU AI Act requires GPAI providers to draw up and maintain extensive technical documentation. The Verifiable Interaction Ledger (VIL) serves as the core of this compliance, providing an immutable, timestamped record of a model's lifecycle, including:
    - **Training Data Summaries**: The VIL can store a cryptographically hashed summary of the datasets used for training, providing a verifiable record without needing to store the data itself. The CitationVerifierV4 and MediaIntegrityAssessor modules provide the tools to generate these summaries, fulfilling the need to document the use of copyright-protected content.
    - **Evaluation and Testing Results**: All outputs from the AIntegrity analysis and monitoring modules are logged to the VIL, creating an auditable history of the model's performance, robustness, and cybersecurity posture.
- **Transparency and Copyright Obligations (Article 53)**: The Act mandates transparency regarding training data, particularly copyrighted material. The VIL, populated by the CitationVerifierV4, provides a mechanism to create the required "sufficiently detailed summary" of training content. Smart contracts on the ledger can enforce policies related to honoring opt-outs (e.g., robots.txt), creating a verifiable compliance trail.
- **Systemic Risk Mitigation (Article 55)**: For GPAI models designated as having systemic risk (those trained with over 10^{25} FLOPs), the Act imposes heightened obligations for model evaluation, risk assessment, and incident reporting. AIntegrity v4.0 is designed for this purpose:
    - The AdversarialThreatMonitor provides the continuous model evaluation and real-time threat detection required.
    - The integrated red-teaming capabilities allow for systematic adversarial testing.
    - The VIL provides an immutable log for reporting serious incidents to the AI Office and national authorities, as required.

Implementation of the NIST AI Risk Management Framework

The NIST AI RMF provides a voluntary but widely adopted structure for managing AI risks

throughout the system lifecycle. AIntegrity v4.0 is not merely a system that can be audited *using* the RMF; it is a platform that *operationalizes* the RMF's core functions, transforming them from a static, document-based exercise into a dynamic, continuous assurance process.

- **Govern**: This function is about establishing a risk management culture and accountability. AIntegrity implements this by allowing AI usage policies and risk thresholds to be encoded directly into the system's configuration and smart contracts. The VIL provides the transparent, auditable foundation required for all governance activities.
- **Map**: This function involves identifying the context and potential impacts of an AI system. The VIL serves as a real-time, comprehensive map of all AI interactions, logging the context, actors, inputs, outputs, and analysis results for every event. This creates a complete inventory for risk assessment.
- **Measure**: This function focuses on assessing and monitoring AI risks. The entire suite of AIntegrity's analysis modules—from the PLIEngine and TrustGradingEngine to the AdversarialThreatMonitor—directly implements the Measure function. They provide a continuous stream of quantitative and qualitative metrics on fairness, bias, accuracy, robustness, and security.
- **Manage**: This function is about allocating resources to treat identified risks. The SentinelEnforcementCore is a direct implementation of the Manage function, providing automated and semi-automated mechanisms to respond to and mitigate risks in real-time (e.g., halting a response, flagging for human review, or isolating a potentially compromised agent).

By embedding these functions into its core architecture, AIntegrity v4.0 enables organizations to move from periodic compliance checks to a state of continuous, verifiable AI assurance. The following table provides a clear mapping between the platform's features and these key regulatory frameworks.

AIntegrity v4.0 Feature/Module	NIST AI RMF Function	EU AI Act Obligation (Article)	Description of Alignment
Verifiable Interaction Ledger (VIL)	Govern, Map	Technical Documentation (Art. 53)	Creates an immutable, auditable inventory of all AI interactions, training data summaries, and evaluation results, forming the foundation for governance and compliance documentation.
CitationVerifierV4	Measure	Copyright Transparency (Art. 53)	Scans AI outputs and training data to produce the "sufficiently detailed summary" of copyrighted content required by the Act.
AdversarialThreatMonitor	Measure, Manage	Systemic Risk Mitigation (Art. 55)	Provides continuous model evaluation, adversarial testing, and real-time incident

AIntegrity v4.0 Feature/Module	NIST AI RMF Function	EU AI Act Obligation (Article)	Description of Alignment
			detection to manage risks for high-impact GPAI models.
ZKP & TEE Integration	Govern, Measure	Accuracy, Robustness, Cybersecurity (Title III)	Provides privacy-preserving proof of compliance with internal policies and the integrity of model execution, enhancing accountability and security.
SentinelEnforcement Core	Manage	Corrective Actions (Title III)	Enforces predefined risk thresholds and policies, enabling automated or semi-automated responses to non-compliant or unsafe AI behavior.
RegulatoryComplianceMapper	Govern	Overall Compliance	Provides a direct bridge between technical audit data and regulatory reporting requirements, streamlining compliance workflows.

Part V: Complete Technical Specification and Implementation

This section provides the complete, implementation-ready technical specification for the AIntegrity v4.0 system. The implementation is presented in Python 3.9+ and builds upon the foundational codebase of AIntegrity v2.1, integrating the new modules and architectural paradigms described in this document.

Core Data Structures and Enumerations

The core data structures are updated to support multimodality and the expanded set of analysis events.

```
# aintegrity/core/data_structures.py

import json
import hashlib
from dataclasses import dataclass, field, asdict
from enum import Enum
```

```
from typing import List, Dict, Any, Optional, Union
import datetime
import uuid

class EventType(Enum):
    """Enumeration of all possible event types in the AIntegrity
log."""
    # Core Interaction
    USER_INPUT = "USER_INPUT"
    MODEL_OUTPUT = "MODEL_OUTPUT"

    # Analysis Events
    LOGICAL_ANALYSIS = "LOGICAL_ANALYSIS"
    VISUAL_CONSISTENCY_ANALYSIS = "VISUAL_CONSISTENCY_ANALYSIS"
    MEDIA_INTEGRITY_ANALYSIS = "MEDIA_INTEGRITY_ANALYSIS"
    COMPLIANCE_ANALYSIS = "COMPLIANCE_ANALYSIS"
    CITATION_ANALYSIS = "CITATION_ANALYSIS"
    SESSION_DRIFT_ANALYSIS = "SESSION_DRIFT_ANALYSIS"
    ADVERSARIAL_THREAT_ANALYSIS = "ADVERSARIAL_THREAT_ANALYSIS"

    # Scoring and Remediation
    TRUST_GRADING = "TRUST_GRADING"
    RECONSTRUCTION = "RECONSTRUCTION"

    # System and Governance
    ENFORCEMENT_ACTION = "ENFORCEMENT_ACTION"
    SYSTEM_STATE = "SYSTEM_STATE"

class ModalityType(Enum):
    """Enumeration for content modalities."""
    TEXT = "text"
    IMAGE = "image"
    AUDIO = "audio"
    VIDEO = "video"
    CODE = "code"

@dataclass
class ContentBlock:
    """Represents a piece of content with a specific modality."""
    modality: ModalityType
    data: Union[str, bytes]  # Text as string, binary data for others
    metadata: Dict[str, Any] = field(default_factory=dict)

    def to_dict(self):
        data_repr = self.data if isinstance(self.data, str) else
self.data.hex()
        return {
            "modality": self.modality.value,
```

```
            "data": data_repr,
            "metadata": self.metadata
        }

@dataclass
class ProofPayload:
    """Container for verifiable computation proofs."""
    proof_type: str  # e.g., "ZKP_SNARK_GROTH16",
"TEE_INTEL_SGX_QUOTE"
    proof_data: Dict[str, Any]
    verification_key_id: Optional[str] = None

@dataclass
class AuditEvent:
    """Represents a single, verifiable event in an interaction
session."""
    event_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    session_id: str = ""
    timestamp_utc: str = field(default_factory=lambda:
datetime.datetime.utcnow().isoformat() + "Z")
    event_type: EventType = EventType.SYSTEM_STATE
    actor_id: str = "system"  # Can be user DID, agent DID, or module
name
    content_blocks: List = field(default_factory=list)
    analysis_payload: Dict[str, Any] = field(default_factory=dict)
    parent_event_id: Optional[str] = None

    # Cryptographic elements added by the VIL
    content_hash: Optional[str] = None
    prev_event_hash: Optional[str] = None
    signature_b64: Optional[str] = None
    proof: Optional[ProofPayload] = None

    def to_canonical_dict(self, include_crypto: bool = False) ->
Dict[str, Any]:
        """Creates a dictionary representation for hashing and
signing."""
        d = {
            "event_id": self.event_id,
            "session_id": self.session_id,
            "timestamp_utc": self.timestamp_utc,
            "event_type": self.event_type.value,
            "actor_id": self.actor_id,
            "content_blocks": [block.to_dict() for block in
self.content_blocks],
            "analysis_payload": self.analysis_payload,
            "parent_event_id": self.parent_event_id,
        }
```

```
        if include_crypto:
            d["content_hash"] = self.content_hash
            d["prev_event_hash"] = self.prev_event_hash
            d["signature_b64"] = self.signature_b64
            if self.proof:
                d["proof"] = asdict(self.proof)
        return d

    def compute_content_hash(self) -> str:
        """Computes the SHA-256 hash of the canonical event
content."""
        # The content hash covers the core data, excluding other
crypto fields
        payload_to_hash = self.to_canonical_dict(include_crypto=False)
        canonical_str = json.dumps(payload_to_hash, sort_keys=True,
separators=(",", ":"))
        return
hashlib.sha256(canonical_str.encode('utf-8')).hexdigest()

@dataclass
class SessionSummary:
    """Cryptographic summary of a completed and sealed session."""
    schema_version: str = "4.0"
    session_id: str
    event_count: int
    start_time_utc: str
    sealed_time_utc: str
    final_event_hash: str
    merkle_root: str
    signing_key_id: str  # e.g., a DID or public key identifier
    tsa_token_rfc3161_b64: Optional[str] = None # Trusted Timestamp
    blockchain_receipt: Optional] = None # e.g., {"tx_hash": "...",
"block_number":...}
```

Verifiable Interaction Ledger (VIL) v4.0

The VerifiableInteractionLedger replaces the VILEngine. It manages the cryptographic lifecycle of events, including signing, hashing, and anchoring to the blockchain.

```
# aintegrity/core/vil.py

import base64
import hashlib
import json
from typing import List, Dict, Any, Optional

from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import ed25519
```

```
# Placeholder for blockchain and TSA client implementations
from.blockchain_client import AbstractBlockchainClient,
MockBlockchainClient
from.tsa_client import AbstractTSAClient, MockTSAClient
from.data_structures import AuditEvent, SessionSummary

class VerifiableInteractionLedger:
    """
    Manages the creation, cryptographic sealing, and decentralized
anchoring of audit events.
    Version: 4.0
    """
    def __init__(self, session_id: str, blockchain_client: Optional =
None, tsa_client: Optional = None):
        self.session_id = session_id
        self.events: List[AuditEvent] =
        self.merkle_leaves: List[str] =
        self.last_event_hash: Optional[str] = None
        self.start_time_utc = datetime.datetime.utcnow().isoformat() +
"Z"

        # Cryptographic key management
        self.private_key = ed25519.Ed25519PrivateKey.generate()
        self.public_key = self.private_key.public_key()
        self.key_id = f"did:key:{self.get_public_key_b58()}"

        # External service clients
        self.blockchain_client = blockchain_client or
MockBlockchainClient()
        self.tsa_client = tsa_client or MockTSAClient()

    def get_public_key_pem(self) -> str:
        return self.public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ).decode('utf-8')

    def get_public_key_b58(self) -> str:
        # Example for did:key format (multicodec prefix for Ed25519 +
raw public key)
        raw_bytes = self.public_key.public_bytes(
            encoding=serialization.Encoding.Raw,
            format=serialization.PublicFormat.Raw
        )
        # 0xed is the multicodec prefix for ed25519-pub
        prefixed_bytes = b'\xed\x01' + raw_bytes
        # A full implementation would use a proper base58btc encoder
```

```
        return base64.b85encode(prefixed_bytes).decode('ascii')

    def _sign_hash(self, data_hash: str) -> str:
        """Signs a hex digest string with the session's private
key."""
        signature = self.private_key.sign(bytes.fromhex(data_hash))
        return base64.b64encode(signature).decode('utf-8')

    def _build_merkle_root(self, leaves: List[str]) -> str:
        """Builds a Merkle tree and returns the root hash."""
        if not leaves:
            return hashlib.sha256(b"").hexdigest()

        current_level = leaves[:]
        while len(current_level) > 1:
            if len(current_level) % 2!= 0:
                current_level.append(current_level[-1]) # Duplicate
last element if odd

            next_level =
            for i in range(0, len(current_level), 2):
                left = current_level[i]
                right = current_level[i+1]
                combined = (left + right).encode('utf-8')

next_level.append(hashlib.sha256(combined).hexdigest())
            current_level = next_level
        return current_level

    def log_event(self, event: AuditEvent) -> AuditEvent:
        """Processes, seals, and logs a single audit event."""
        event.session_id = self.session_id

        # 1. Chain hash
        event.prev_event_hash = self.last_event_hash

        # 2. Content hash
        event.content_hash = event.compute_content_hash()

        # 3. Signature
        # The signature covers the hash of the canonical
representation of the event *header*
        # This is a simplified JWS-like approach
        header_plus_payload_dict = {
            "header": {
                "alg": "EdDSA",
                "kid": self.key_id
            },
```

```
            "payload": event.to_canonical_dict(include_crypto=False)
        }
        canonical_header_payload_str =
json.dumps(header_plus_payload_dict, sort_keys=True, separators=(",",
":"))
        message_to_sign_hash =
hashlib.sha256(canonical_header_payload_str.encode('utf-8')).hexdigest
()
        event.signature_b64 = self._sign_hash(message_to_sign_hash)

        # 4. Update ledger state
        self.events.append(event)
        self.merkle_leaves.append(event.content_hash)

        # The hash for the next event's prev_event_hash is the hash of
the full signed event record
        full_event_dict = event.to_canonical_dict(include_crypto=True)
        canonical_full_event_str = json.dumps(full_event_dict,
sort_keys=True, separators=(",", ":"))
        self.last_event_hash =
hashlib.sha256(canonical_full_event_str.encode('utf-8')).hexdigest()

        return event

    def seal_and_anchor_session(self) -> SessionSummary:
        """Finalizes the session, generates the summary, and anchors
it to the blockchain."""
        if not self.events:
            raise ValueError("Cannot seal an empty session.")

        sealed_time_utc = datetime.datetime.utcnow().isoformat() + "Z"
        merkle_root = self._build_merkle_root(self.merkle_leaves)

        summary = SessionSummary(
            session_id=self.session_id,
            event_count=len(self.events),
            start_time_utc=self.start_time_utc,
            sealed_time_utc=sealed_time_utc,
            final_event_hash=self.last_event_hash,
            merkle_root=merkle_root,
            signing_key_id=self.key_id
        )

        # Anchor the summary hash
        summary_dict = asdict(summary)
        summary_str = json.dumps(summary_dict, sort_keys=True,
separators=(",", ":"))
        summary_hash =
```

```
hashlib.sha256(summary_str.encode('utf-8')).hexdigest()

        # 1. Get Trusted Timestamp
        summary.tsa_token_rfc3161_b64 =
self.tsa_client.get_timestamp_token(summary_hash.encode('utf-8'))

        # 2. Anchor to Blockchain
        summary.blockchain_receipt =
self.blockchain_client.anchor_hash(summary_hash)

        return summary
```

Multimodal Verification Engine

This new engine contains modules for handling non-textual data, providing critical verification for modern AI systems.

```
# aintegrity/modules/multimodal_verifier.py

from typing import List, Dict, Any
from PIL import Image
import io
import imagehash # For perceptual hashing (pHash)
import torch
from transformers import CLIPProcessor, CLIPModel

from aintegrity.core.data_structures import ContentBlock, ModalityType

class VisualConsistencyVerifier:
    """
    Verifies semantic consistency between image and text content
blocks using CLIP.
    Version: 1.0
    """
    def __init__(self, model_name: str =
"openai/clip-vit-large-patch14"):
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.model =
CLIPModel.from_pretrained(model_name).to(self.device)
        self.processor = CLIPProcessor.from_pretrained(model_name)

    def verify(self, content_blocks: List) -> Dict[str, Any]:
        """
        Calculates the consistency score between the first image and
first text block.
        """
        image_block = next((b for b in content_blocks if b.modality ==
ModalityType.IMAGE), None)
```

```
        text_block = next((b for b in content_blocks if b.modality ==
ModalityType.TEXT), None)

        if not image_block or not text_block:
            return {"status": "SKIPPED", "reason": "Missing required
image or text modality."}

        try:
            image = Image.open(io.BytesIO(image_block.data))
            text = text_block.data

            inputs = self.processor(text=[text], images=image,
return_tensors="pt", padding=True).to(self.device)

            with torch.no_grad():
                outputs = self.model(**inputs)

            logits_per_image = outputs.logits_per_image
            probability = logits_per_image.softmax(dim=1).item()

            return {
                "status": "COMPLETED",
                "consistency_score": probability,
                "model_used": self.model.config.name_or_path
            }
        except Exception as e:
            return {"status": "ERROR", "reason": str(e)}

class MediaIntegrityAssessor:
    """
    Assesses the integrity of media files using perceptual hashing.
    Version: 1.0
    """
    def assess(self, content_block: ContentBlock) -> Dict[str, Any]:
        """Generates a perceptual hash for a given media content
block."""
        if content_block.modality not in:
            return {"status": "SKIPPED", "reason": f"Unsupported
modality for pHash: {content_block.modality.value}"}

        try:
            if content_block.modality == ModalityType.IMAGE:
                image = Image.open(io.BytesIO(content_block.data))
                # Average hashing is simple and fast
                phash = str(imagehash.average_hash(image))
                return {
                    "status": "COMPLETED",
                    "perceptual_hash": phash,
```

```
                    "hash_algorithm": "average_hash"
                }
            # Placeholder for video hashing
            elif content_block.modality == ModalityType.VIDEO:
                 return {"status": "NOT_IMPLEMENTED", "reason": "Video
perceptual hashing is not yet implemented."}

        except Exception as e:
            return {"status": "ERROR", "reason": str(e)}
```

Dynamic Trust Scoring Engine

The TrustGradingEngine is updated to version 4.0, now incorporating the TrustDecayModel.

```
# aintegrity/modules/trust_grader.py

import math
import time
from typing import Dict, Any, Optional

class TrustDecayModel:
    """Models the temporal decay of a trust score."""
    def __init__(self, decay_rate: float = 0.01, min_score: float =
0.2, midpoint_days: float = 30.0):
        self.k = decay_rate
        self.t_mid = midpoint_days * 86400  # Convert days to seconds
        self.t_min = min_score
        self.t_max = 1.0
        self.last_update_timestamp = time.time()

    def update_score(self, new_score: float):
        """Reset the decay model with a new score."""
        self.t_max = new_score
        self.last_update_timestamp = time.time()

    def get_current_score(self) -> float:
        """Calculate the decayed score based on elapsed time."""
        elapsed_time = time.time() - self.last_update_timestamp

        # Logistic decay function
        denominator = 1 + math.exp(self.k * (elapsed_time -
self.t_mid))
        decayed_score = self.t_min + (self.t_max - self.t_min) /
denominator

        return max(self.t_min, decayed_score)

class TrustGradingEngineV4:
```

```
    """
    Calculates a dynamic, multi-dimensional trust score for an AI
agent or session.
    Version: 4.0
    """
    def __init__(self, agent_id: str, initial_weights: Optional] =
None):
        self.agent_id = agent_id
        self.decay_model = TrustDecayModel()
        self.weights = initial_weights or {
            "logical_consistency": 0.25,
            "factual_accuracy": 0.20,
            "visual_consistency": 0.15,
            "behavioral_stability": 0.20,
            "adversarial_resistance": 0.20
        }

    def calculate_trust_score(self, analysis_results: Dict[str, Any])
-> Dict[str, Any]:
        """Calculates a comprehensive trust score from various
analysis module outputs."""

        components = {
            "logical_consistency":
analysis_results.get("logical_analysis", {}).get("consistency_score",
1.0),
            "factual_accuracy":
analysis_results.get("citation_analysis",
{}).get("verifiability_score", 1.0),
            "visual_consistency":
analysis_results.get("visual_consistency",
{}).get("consistency_score", 1.0),
            "behavioral_stability": 1.0 -
analysis_results.get("session_drift", {}).get("max_severity", 0.0),
            "adversarial_resistance": 1.0 -
analysis_results.get("adversarial_threat", {}).get("threat_level",
0.0)
        }

        # Calculate weighted overall score
        weighted_sum = sum(self.weights[k] * v for k, v in
components.items() if v is not None)
        total_weight = sum(self.weights[k] for k, v in
components.items() if v is not None)

        overall_score = weighted_sum / total_weight if total_weight >
0 else 0.0
```

```
        # Update the decay model with this new score
        self.decay_model.update_score(overall_score)

        return {
            "agent_id": self.agent_id,
            "overall_score_instantaneous": overall_score,
            "overall_score_decayed":
self.decay_model.get_current_score(),
            "components": components,
            "weights": self.weights,
            "calculation_timestamp_utc":
datetime.datetime.utcnow().isoformat() + "Z"
        }

    def get_current_trust_score(self) -> float:
        """Returns the current, time-decayed trust score."""
        return self.decay_model.get_current_score()
```

Real-Time Adversarial Monitoring

The AdversarialThreatMonitor is a new, critical module for proactive defense.

```
# aintegrity/modules/threat_monitor.py

import numpy as np
from typing import Dict, Any, List

# Placeholder for a real SHAP explainer
class MockShapExplainer:
    def explain(self, data):
        # In a real implementation, this would return actual SHAP
values
        return np.random.rand(len(data.split()))

class AdversarialThreatMonitor:
    """
    Monitors AI interactions in real-time for adversarial attacks and
anomalies.
    Version: 1.0
    """
    def __init__(self, baseline_data: List[str]):
        self.psi_baseline_dist =
self._calculate_distribution(baseline_data)
        self.xai_explainer = MockShapExplainer()
        self.shap_baseline_dist =
self._calculate_shap_distribution(baseline_data)

    def _calculate_distribution(self, data: List[str], num_bins: int =
```

```
10) -> np.ndarray:
        """Calculates a simple feature distribution (e.g., based on
length)."""
        lengths = [len(s) for s in data]
        if not lengths:
            return np.zeros(num_bins)
        hist, _ = np.histogram(lengths, bins=num_bins, range=(0,
max(lengths) if lengths else 1))
        return (hist / len(lengths)).astype(float)

    def _calculate_shap_distribution(self, data: List[str], num_bins:
int = 10) -> np.ndarray:
        """Calculates a distribution of SHAP value means."""
        shap_means = [np.mean(self.xai_explainer.explain(s)) for s in
data]
        if not shap_means:
            return np.zeros(num_bins)
        hist, _ = np.histogram(shap_means, bins=num_bins, range=(0,
1))
        return (hist / len(shap_means)).astype(float)

    def _calculate_psi(self, baseline_dist: np.ndarray, current_dist:
np.ndarray) -> float:
        """Calculates the Population Stability Index."""
        # Add a small epsilon to avoid division by zero
        epsilon = 1e-10
        baseline_dist = np.where(baseline_dist == 0, epsilon,
baseline_dist)
        current_dist = np.where(current_dist == 0, epsilon,
current_dist)

        psi_value = np.sum((current_dist - baseline_dist) *
np.log(current_dist / baseline_dist))
        return psi_value

    def monitor(self, current_batch: List[str]) -> Dict[str, Any]:
        """Monitors a batch of new data for drift and XAI
anomalies."""
        if not current_batch:
            return {"status": "SKIPPED", "reason": "Empty batch."}

        # Data Drift (PSI)
        current_dist_psi = self._calculate_distribution(current_batch)
        psi_score = self._calculate_psi(self.psi_baseline_dist,
current_dist_psi)

        # XAI Anomaly (PSI on SHAP values)
        current_dist_shap =
```

```
self._calculate_shap_distribution(current_batch)
        xai_anomaly_score =
self._calculate_psi(self.shap_baseline_dist, current_dist_shap)

        # Simple threat level determination
        threat_level = 0.0
        if psi_score > 0.2 or xai_anomaly_score > 0.2:
            threat_level = 1.0 # Significant drift/anomaly
        elif psi_score > 0.1 or xai_anomaly_score > 0.1:
            threat_level = 0.5 # Moderate drift/anomaly

        return {
            "status": "COMPLETED",
            "data_drift_psi": psi_score,
            "xai_anomaly_psi": xai_anomaly_score,
            "threat_level": threat_level,
            "is_alert": threat_level > 0.5
        }
```

AIntegrity Core Orchestrator v4.0

The main orchestrator is upgraded to integrate the new modules and workflows.

```
# aintegrity/core/orchestrator.py

from.data_structures import AuditEvent, EventType, ContentBlock,
ModalityType
from.vil import VerifiableInteractionLedger
from aintegrity.modules.pli_engine import PLIEngineV4 # Assuming
updated PLI Engine
from aintegrity.modules.multimodal_verifier import
VisualConsistencyVerifier, MediaIntegrityAssessor
from aintegrity.modules.compliance_scanner import
ComplianceScanModuleV2 # Assuming updated module
from aintegrity.modules.citation_verifier import CitationVerifierV4 #
Assuming updated module
from aintegrity.modules.drift_detector import SessionDriftDetectorV4 #
Assuming updated module
from aintegrity.modules.threat_monitor import AdversarialThreatMonitor
from aintegrity.modules.trust_grader import TrustGradingEngineV4
from aintegrity.modules.reconstruction import ReconstructionAdvisorV2
# Assuming updated module
from aintegrity.modules.enforcement import SentinelEnforcementCoreV2 #
Assuming updated module

class AIntegrityCoreV4:
    """
    Main orchestrator for the AIntegrity v4.0 framework.
```

```
    """
    def __init__(self, session_id: str, baseline_data_for_monitor:
List[str]):
        self.vil = VerifiableInteractionLedger(session_id)

        # Initialize all analysis modules
        self.pli_engine = PLIEngineV4()
        self.visual_verifier = VisualConsistencyVerifier()
        self.media_assessor = MediaIntegrityAssessor()
        self.compliance_scanner = ComplianceScanModuleV2()
        self.citation_verifier = CitationVerifierV4()
        self.drift_detector = SessionDriftDetectorV4()
        self.threat_monitor =
AdversarialThreatMonitor(baseline_data_for_monitor)
        self.trust_grader =
TrustGradingEngineV4(agent_id="default_agent")
        self.reconstructor = ReconstructionAdvisorV2()
        self.sentinel = SentinelEnforcementCoreV2()

        print("AIntegrity Core v4.0 initialized.")

    def process_interaction_turn(self, user_content: List,
model_content: List, parent_event_id: Optional[str]) -> str:
        """Processes a single turn of a multimodal conversation."""

        # 1. Log User Input
        user_event = AuditEvent(
            event_type=EventType.USER_INPUT,
            actor_id="user_did_placeholder",
            content_blocks=user_content,
            parent_event_id=parent_event_id
        )
        logged_user_event = self.vil.log_event(user_event)

        # 2. Log Model Output
        model_event = AuditEvent(
            event_type=EventType.MODEL_OUTPUT,
            actor_id="model_did_placeholder",
            content_blocks=model_content,
            parent_event_id=logged_user_event.event_id
        )
        logged_model_event = self.vil.log_event(model_event)

        # 3. Run Real-time Adversarial Monitoring on inputs
        text_inputs =
        threat_results = self.threat_monitor.monitor(text_inputs)
        threat_event = AuditEvent(
            event_type=EventType.ADVERSARIAL_THREAT_ANALYSIS,
```

```
            analysis_payload=threat_results,
            parent_event_id=logged_user_event.event_id
        )
        self.vil.log_event(threat_event)

        # 4. Run Core Analysis Modules on model output
        analysis_results = {}
        text_outputs =

        # Run analyses and store results
        analysis_results["logical_analysis"] =
self.pli_engine.analyze(text_outputs)
        analysis_results["visual_consistency"] =
self.visual_verifier.verify(model_content)
        #... other analyses (compliance, citation, drift) would run
here...

        # 5. Grade Trust
        trust_score_result =
self.trust_grader.calculate_trust_score(analysis_results)
        trust_event = AuditEvent(
            event_type=EventType.TRUST_GRADING,
            analysis_payload=trust_score_result,
            parent_event_id=logged_model_event.event_id
        )
        self.vil.log_event(trust_event)

        # 6. Sentinel Enforcement
        all_findings = {**analysis_results, "threat": threat_results,
"trust": trust_score_result}
        enforcement_decision = self.sentinel.enforce(all_findings)

        if enforcement_decision["final_decision"]!= "PASS":
            enforcement_event = AuditEvent(
                event_type=EventType.ENFORCEMENT_ACTION,
                analysis_payload=enforcement_decision,
                parent_event_id=logged_model_event.event_id
            )
            self.vil.log_event(enforcement_event)
            print(f"SENTINEL ACTION:
{enforcement_decision['final_decision']} - REASON:
{enforcement_decision['rationale']}")

        return logged_model_event.event_id

    def finalize_session(self):
        """Seals the session and anchors it."""
        summary = self.vil.seal_and_anchor_session()
```

```
        print("\n--- SESSION FINALIZED ---")
        print(json.dumps(asdict(summary), indent=2))
        return summary

# Example Usage
if __name__ == '__main__':
    # Initialize with some baseline text data for the monitor
    baseline_texts =
    core = AIntegrityCoreV4(session_id=str(uuid.uuid4()),
baseline_data_for_monitor=baseline_texts)

    # --- Turn 1 ---
    print("\n--- Processing Turn 1 ---")
    user_turn_1 =
    model_turn_1 =
    last_event_id = core.process_interaction_turn(user_turn_1,
model_turn_1, parent_event_id=None)

    # --- Turn 2 (with potential adversarial input) ---
    print("\n--- Processing Turn 2 ---")
    user_turn_2 =
    model_turn_2 =
    last_event_id = core.process_interaction_turn(user_turn_2,
model_turn_2, parent_event_id=last_event_id)

    # --- Finalize ---
    core.finalize_session()
```

Works cited

1. NIST AI Risk Management Framework: A tl;dr - Wiz, https://www.wiz.io/academy/nist-ai-risk-management-framework 2. EU Artificial Intelligence Act | Up-to-date developments and analyses of the EU AI Act, https://artificialintelligenceact.eu/ 3. Art. 25 GDPR – Data protection by design and by default, https://gdpr-info.eu/art-25-gdpr/ 4. How to Demonstrate Compliance With GDPR Article 25 | ISMS.online, https://www.isms.online/general-data-protection-regulation-gdpr/gdpr-article-25-compliance/ 5. GDPR Article 25 - Imperva, https://www.imperva.com/learn/data-security/gdpr-article-25/ 6. Deceptive Patterns - Laws - GDPR - Article 25, https://www.deceptive.design/laws/gdpr-article-25 7. Blockchain-Based Security Audit Trails: Definition, Examples, and ..., https://www.graphapp.ai/engineering-glossary/cloud-computing/blockchain-based-security-audit-trails 8. Using Blockchain Ledgers to Record AI Decisions in IoT - MDPI, https://www.mdpi.com/2624-831X/6/3/37 9. Redis persistence | Docs, https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/ 10. Append-Only Logs: The Immutable Diary of Data | by komal shehzadi | Medium, https://medium.com/@komalshehzadi/append-only-logs-the-immutable-diary-of-data-58c36a871

c7c 11. The Log: What every software engineer should know about real-time data's unifying abstraction, https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying 12. Efficient Data Retrieval from a Secure, Durable, Append-Only Log - Sam Kumar, https://www.samkumar.org/projects/gdpfs.pdf 13. Hyperledger Fabric Vs Ethereum: A Comprehensive Comparison | by Spydra - Medium, https://medium.com/coinmonks/hyperledger-fabric-vs-ethereum-a-comprehensive-comparison-d557d2f86231 14. Best Comparison of Ethereum, Hyperledger Fabric and Corda - NASSCOM Community, https://community.nasscom.in/communities/blockchain/best-comparison-ethereum-hyperledger-fabric-and-corda-0 15. Hyperledger vs Ethereum: A Clash of Two Emerging Technologies - 101 Blockchains, https://101blockchains.com/hyperledger-vs-ethereum-2/ 16. Scalability and Efficiency Analysis of Hyperledger Fabric and Private Ethereum in Smart Contract Execution - MDPI, https://www.mdpi.com/2073-431X/14/4/132 17. Hyperledger vs Ethereum in Blockchain - GeeksforGeeks, https://www.geeksforgeeks.org/computer-networks/blockchain-hyperledger-vs-ethereum/ 18. Ethereum Vs. Hyperledger: Key Differences | Shardeum, https://shardeum.org/blog/ethereum-vs-hyperledger/ 19. Zero-Knowledge Proof – How It Works - Hacken.io, https://hacken.io/discover/zero-knowledge-proof/ 20. Tutorial 5: Zero Knowledge Proofs - sCrypt, https://docs.scrypt.io/bsv-docs/tutorials/zkp/ 21. What are Zero Knowledge Proofs: A Beginner's Guide to Creating ..., https://0xazan.hashnode.dev/what-are-zero-knowledge-proofs-a-beginners-guide-to-creating-your-first-zk-proof-with-circom-and-snarkjs-on-scroll-network 22. JavaScript tutorial for Zero-Knowledge Proofs Using SnarkJS and Circom | HackerNoon, https://hackernoon.com/javascript-tutorial-for-zero-knowledge-proofs-using-snarkjs-and-circom 23. A curated list of awesome things related to learning Zero-Knowledge Proofs (ZKP). - GitHub, https://github.com/matter-labs/awesome-zero-knowledge-proofs 24. Trusted Execution Environment (TEE) | Microsoft Learn, https://learn.microsoft.com/en-us/azure/confidential-computing/trusted-execution-environment 25. Trusted execution environment - Wikipedia, https://en.wikipedia.org/wiki/Trusted_execution_environment 26. A Comprehensive Analysis of Trusted Execution Environments - ResearchGate, https://www.researchgate.net/publication/363106383_A_Comprehensive_Analysis_of_Trusted_Execution_Environments 27. ZKPs & TEEs: Can These Be Combined? | Metaverse Post, https://mpost.io/zkps-tees-can-these-be-combined/ 28. Confidential Kubernetes: Use Confidential Virtual Machines and Enclaves to improve your cluster security, https://kubernetes.io/blog/2023/07/06/confidential-kubernetes/ 29. What is the difference between trusted computing and confidential computing? - Stack Overflow, https://stackoverflow.com/questions/63335341/what-is-the-difference-between-trusted-computing-and-confidential-computing 30. Distill CLIP (DCLIP): Enhancing Image-Text Retrieval via Cross-Modal Transformer Distillation - arXiv, https://arxiv.org/html/2505.21549v2 31. (PDF) Quantifying Interpretability in CLIP Models with Concept Consistency - ResearchGate, https://www.researchgate.net/publication/389894333_Quantifying_Interpretability_in_CLIP_Models_with_Concept_Consistency 32. arxiv.org, https://arxiv.org/html/2503.05303v1 33. CLIP Behaves like a Bag-of-Words Model Cross-modally but not Uni-modally - arXiv, https://arxiv.org/html/2502.03566v2 34. CLIP Behaves like a Bag-of-Words Model Cross-modally but not Uni-modally - ChatPaper, https://chatpaper.com/paper/105754 35. CLIP Behaves like a Bag-of-Words Model Cross-modally but not Uni-modally,

https://www.researchgate.net/publication/388790993_CLIP_Behaves_like_a_Bag-of-Words_Model_Cross-modally_but_not_Uni-modally 36. Perceptual hashing - Wikipedia, https://en.wikipedia.org/wiki/Perceptual_hashing 37. Perceptual Video Hashing for Content Identification and Authentication - ResearchGate, https://www.researchgate.net/publication/321202433_Perceptual_Video_Hashing_for_Content_Identification_and_Authentication 38. (PDF) From Image Hashing to Video Hashing - ResearchGate, https://www.researchgate.net/publication/220988836_From_Image_Hashing_to_Video_Hashing 39. Blockchain for video watermarking: An enhanced copyright protection approach for video forensics based on perceptual hash function - PubMed Central, https://pmc.ncbi.nlm.nih.gov/articles/PMC11495578/ 40. How do hashing techniques accelerate audio search? - Milvus, https://milvus.io/ai-quick-reference/how-do-hashing-techniques-accelerate-audio-search 41. What role do spectrograms play in audio analysis and search? - Milvus, https://milvus.io/ai-quick-reference/what-role-do-spectrograms-play-in-audio-analysis-and-search 42. (PDF) Perceptual Audio Hashing Functions - ResearchGate, https://www.researchgate.net/publication/26531832_Perceptual_Audio_Hashing_Functions 43. PERMUTATION GROUPING: INTELLIGENT HASH FUNCTION DESIGN FOR AUDIO & IMAGE RETRIEVAL - Google Research, https://research.google.com/pubs/archive/34409.pdf 44. Spectrogram-based Efficient Perceptual Hashing Scheme for Speech Identification - International Journal of Network Security, http://ijns.jalaxy.com.tw/contents/ijns-v21-n2/ijns-2019-v21-n2-p259-268.pdf 45. Classical Mathematical Models for Description and Prediction of ..., https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1003800 46. Lessons from mathematically modeling the NF-κB pathway - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC3343698/ 47. (PDF) Trust Decay-Based Temporal Learning for Dynamic Recommender Systems With Concept Drift Adaptation - ResearchGate, https://www.researchgate.net/publication/393042281_Trust_Decay-Based_Temporal_Learning_for_Dynamic_Recommender_Systems_with_Concept_Drift_Adaptation 48. Classical Mathematical Models for Description and Prediction of Experimental Tumor Growth | PLOS Computational Biology, https://journals.plos.org/ploscompbiol/article%3Fid%3D10.1371/journal.pcbi.1003800 49. What Is Model Drift? | IBM, https://www.ibm.com/think/topics/model-drift 50. Model Drift: What It Is & How To Avoid Drift in AI/ML Models - Splunk, https://www.splunk.com/en_us/blog/learn/model-drift.html 51. What is Model Drift in Machine Learning? | Domino Data Lab, https://domino.ai/data-science-dictionary/model-drift 52. Understanding Model Drift and Data Drift in LLMs (2025 Guide) | Generative AI Collaboration Platform - Orq.ai, https://orq.ai/blog/model-vs-data-drift 53. Understanding and Mitigating Model Drift in Machine Learning - ProjectPro, https://www.projectpro.io/article/model-drift-in-machine-learning/871 54. How to Monitor LLMOps Performance with Drift Monitoring | Fiddler AI Blog, https://www.fiddler.ai/blog/how-to-monitor-llmops-performance-with-drift 55. Addressing Data Drift in Large Language Models (LLMs) - DEV Community, https://dev.to/kapusto/addressing-data-drift-in-large-language-models-llms-gpa 56. What are adversarial attacks in anomaly detection? - Milvus, https://milvus.io/ai-quick-reference/what-are-adversarial-attacks-in-anomaly-detection 57. What Is Adversarial AI in Machine Learning? - Palo Alto Networks, https://www.paloaltonetworks.com/cyberpedia/what-are-adversarial-attacks-on-AI-Machine-Lear

ning 58. What Is Anomaly Detection? - CrowdStrike, https://www.crowdstrike.com/en-us/cybersecurity-101/next-gen-siem/anomaly-detection/ 59. Robust Intrusion Detection System with Explainable Artificial ... - arXiv, https://arxiv.org/pdf/2503.05303 60. Robust Intrusion Detection System with Explainable Artificial Intelligence - arXiv, https://arxiv.org/abs/2503.05303 61. Explainable Intrusion Detection Systems (X-IDS): A Survey of Current Methods, Challenges, and Opportunities - ResearchGate, https://www.researchgate.net/publication/365103539_Explainable_Intrusion_Detection_Systems_X-IDS_A_Survey_of_Current_Methods_Challenges_and_Opportunities 62. Detecting Adversarial DDoS Attacks in Software- Defined Networking Using Deep Learning Techniques and Adversarial Training - ResearchGate, https://www.researchgate.net/publication/354414532_Detecting_Adversarial_DDoS_Attacks_in_Software-_Defined_Networking_Using_Deep_Learning_Techniques_and_Adversarial_Training 63. Machine Learning Mar 2025 - arXiv, http://arxiv.org/list/cs.LG/2025-03?skip=1400&show=1000 64. TXT file - Nicholas Carlini, https://nicholas.carlini.com/writing/2019/advex_papers.txt 65. EU rules on general-purpose AI models start to apply, bringing more ..., https://digital-strategy.ec.europa.eu/en/news/eu-rules-general-purpose-ai-models-start-apply-bringing-more-transparency-safety-and-accountability 66. EU AI Act: European Commission Publishes General-Purpose AI Code of Practice | Insights, https://www.jonesday.com/en/insights/2025/08/eu-ai-act-european-commission-publishes-generalpurpose-ai-code-of-practice 67. EU AI Act: Key Compliance Considerations Ahead of August 2025 | Insights, https://www.gtlaw.com/en/insights/2025/7/eu-ai-act-key-compliance-considerations-ahead-of-august-2025 68. Latest wave of obligations under the EU AI Act take effect: Key considerations | DLA Piper, https://www.dlapiper.com/en-hk/insights/publications/2025/08/latest-wave-of-obligations-under-the-eu-ai-act-take-effect 69. AI Act | Shaping Europe's digital future - European Union, https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai 70. EU Artificial Intelligence Act: A General Overview, https://www.curtis.com/our-firm/news/eu-artificial-intelligence-act-a-general-overview 71. No 'Stop the Clock' For the EU AI Act (and a belated General-Purpose AI Code of Practice): What Does This Mean to You? - Kslaw.com, https://www.kslaw.com/news-and-insights/no-stop-the-clock-for-the-eu-ai-act-and-a-belated-general-purpose-ai-code-of-practice-what-does-this-mean-to-you 72. EU AI Act News: Rules on General-Purpose AI Start Applying, Guidelines and Template for Summary of Training Data Finalized - Mayer Brown, https://www.mayerbrown.com/en/insights/publications/2025/08/eu-ai-act-news-rules-on-general-purpose-ai-start-applying-guidelines-and-template-for-summary-of-training-data-finalized 73. The EU Commission Publishes General-Purpose AI Code of Practice: Compliance Obligations Begin August 2025 - Nelson Mullins, https://www.nelsonmullins.com/insights/alerts/privacy_and_data_security_alert/all/the-eu-commission-publishes-general-purpose-ai-code-of-practice-compliance-obligations-begin-august-2025 74. Artificial Intelligence Risk Management Framework (AI RMF 1.0) | NIST, https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10 75. AI Risk Management Framework | NIST, https://www.nist.gov/itl/ai-risk-management-framework 76. NIST Risk Management Framework | CSRC, https://csrc.nist.gov/projects/risk-management 77. NIST AI Risk Management Framework: A simple guide to smarter AI governance - Diligent, https://www.diligent.com/resources/blog/nist-ai-risk-management-framework